

Aula 2 - Manipulação de Arquivos

1. Conceito de Arquivo

Um arquivo é uma coleção de dados armazenados permanentemente em um dispositivo de armazenamento (HD, SSD etc.).

Existem dois tipos principais:

Arquivos de Texto

- Armazenam dados em formato legível por humanos.
- Exemplo: .txt, .csv, .json, .xml

Arquivos Binários

- Armazenam dados em formato binário (0 e 1).
- Não são legíveis diretamente.
- Exemplo: .exe, .jpg, .zip, .mp3

- Operações básicas (abertura, leitura, escrita, fechamento)
- Arquivos sequenciais vs. aleatórios

Manipular arquivos significa permitir que um programa:

- Acesse dados persistentes em disco
- Leia e/ou grave informações
- Controle quando e como esses dados são acessados

Isso é feito por meio de **operações básicas** e de **modelos de acesso**.

Operações básicas

Abertura de arquivos (open)

É o momento em que o programa:

- Localiza o arquivo no sistema
- Solicita permissão (leitura, escrita ou ambos)
- Cria uma **associação** entre o arquivo físico e o programa (*stream*)

É feito da seguinte forma:



Sem abrir, **nenhuma operação é possível**.

Leitura (read)

Consiste em:

- Trazer dados do arquivo para a memória
- Entregar os dados ao programa

Características:

- Pode ser caractere a caractere, linha a linha ou em blocos
- Normalmente usa buffer para eficiência

Escrita (write)

Consiste em:

- Enviar dados do programa para o arquivo
- Armazenar de forma permanente no disco

Características:

- Dados passam primeiro por um **buffer**
- Só chegam ao disco após flush() ou close()

Fechamento (close)

Finaliza o acesso ao arquivo:

- Libera recursos do sistema operacional
- Garante que todos os dados pendentes sejam gravados

A operação de fechamento é essencial, pois um arquivo aberto e não fechado pode causar:

- Vazamento de recursos
- Dados perdidos
- Arquivo corrompido

Arquivos sequenciais vs. aleatórios

Essa distinção define **como os dados são acessados** dentro do arquivo.

1 - Arquivos sequenciais

Características

- Leitura ocorre **do início ao fim**
- Não é possível “pular” diretamente para um ponto específico
- Acesso ocorre na **ordem dos dados**

Exemplo típico

- Arquivos texto (.txt)
- Logs

[Registro 1] → [Registro 2] → [Registro 3]

Para acessar o registro 3, é preciso passar pelos anteriores.

- **Exemplo em Java de leitura de arquivos - Sequencial:**

```
BufferedReader reader = new BufferedReader(new FileReader("dados.txt"));
```

Como entender o que é um registro?

Registro não é um arquivo inteiro.

Registro é uma *unidade de informação* dentro de um arquivo.

No contexto de arquivos

Um **registro** é:

- Um **conjunto de dados relacionados**
 - Tratado como uma **unidade lógica**
 - Armazenado **dentro de um arquivo**
 - É um **bloco organizado de dados**
-

Exemplo de arquivo texto (sequencial)

Arquivo alunos.txt:

```
123;João;8.5  
124;Maria;9.2  
125;Ana;7.8
```

- Cada **linha** → **1 registro**
- Cada parte separada por ; → **campo**

Para o programa que lerá o arquivo, cada linha representa **um aluno**.

Arquivos aleatórios (acesso direto)

Características

- Permitem acessar **qualquer posição do arquivo**
- Baseados em **offsets** (posição em bytes)
- Mais complexos, porém mais eficientes em certos cenários

Exemplo típico

- Bancos de dados simples
- Arquivos binários estruturados
- Índices

[Registro 1] [Registro 2] [Registro 3]
↑ acesso direto

Exemplo em Java (conceitual)

- **Exemplo em Java de leitura de arquivos - Aleatório:**

```
RandomAccessFile raf = new RandomAccessFile("dados.dat", "rw");
raf.seek(100);
```

Comparação entre arquivos sequenciais e aleatórios

Aspecto	Sequencial	Aleatório
Ordem	Fixa	Livre
Acesso direto	Não	Sim
Simplicidade	Alta	Média
Performance	Boa para leitura linear	Melhor para buscas pontuais

Exemplo com arquivo binário (aleatório)

Um registro pode ter **tamanho fixo**:

[ID (4 bytes)][Nota (4 bytes)] [Nome (20 bytes)]

Cada bloco desse é um registro.

Isso permite acesso direto:

registro 3 = posição 3 × tamanho do registro

Por que “registro” é importante?

Porque ele:

- Define **como os dados estão organizados**
 - Permite leitura lógica (“leia um aluno”, não “leia bytes”)
 - É base para **acesso sequencial e aleatório**
-

Exemplo simples com arquivos (Java)

```
FileWriter writer = new FileWriter("dados.txt");  
writer.write("Olá");  
// esqueceu writer.close().
```

O arquivo continua **aberto**, mesmo após o uso.

Isso é um **vazamento de recurso**.

Por que isso é um problema?

Porque:

- O sistema tem **limite** de arquivos abertos
 - Recursos não liberados **não podem ser reutilizados**
 - O programa pode falhar após algum tempo
 - Em casos graves, **o sistema inteiro pode ficar instável**
-

Vazamento de memória × vazamento de recursos

- **Vazamento de memória:** memória alocada que não é mais usada
- **Vazamento de recursos:** recursos do SO não liberados

➡ Em Java, mesmo com garbage collector, **recursos externos precisam ser fechados manualmente**.

Como evitar vazamento de recursos?

```
try (BufferedWriter writer = new BufferedWriter(new FileWriter("dados.txt"))) {  
    writer.write("Olá");  
} // close()
```

Portanto, vazamento de recursos ocorre quando um programa não libera recursos do sistema após o uso, causando desperdício e possíveis falhas.

Quando usar Arquivos?

1. Quando os dados são simples e pequenos

Cenário

- Poucos registros (dezenas ou centenas)
- Estrutura fixa
- Sem consultas complexas

Exemplo

- Arquivo de configuração
- Lista simples de usuários
- Parâmetros de sistema

Melhor alternativa

- Arquivo texto (.txt, .json, .csv)
- Arquivo binário simples

➡ usar banco seria **complexidade desnecessária**.

2 quando não há concorrência

Cenário

- Apenas **um usuário**
- Apenas **um processo**
- Sem acesso simultâneo

Exemplo

- Aplicação desktop local
- Programa acadêmico
- Script de automação

Melhor alternativa

- Arquivo sequencial ou binário estruturado

➡ Banco resolve concorrência — se não há concorrência, ele não agrega valor.

3 Sistemas embarcados ou com poucos recursos

Cenário

- Pouca memória

- Pouco armazenamento
- CPU limitada

Exemplo

- Microcontroladores
- Dispositivos IoT
- Equipamentos industriais

Melhor alternativa

- Arquivos binários
- Memória flash estruturada

➡ Banco de dados é pesado para esse contexto.

4 quando o desempenho bruto é crítico

Cenário

- Escrita/leitura extremamente rápida
- Latência mínima
- Formato customizado

Exemplo

- Jogos (save game)
- Sistemas de tempo real
- Logs de alto desempenho

Melhor alternativa

- Arquivo binário estruturado

➡ O overhead do banco pode ser maior que o ganho.

5 quando os dados são temporários

Cenário

- Dados descartáveis
- Não precisam persistir por muito tempo

Exemplo

- Cache
- Arquivos temporários

- Processamento intermediário

Melhor alternativa

- Memória
- Arquivos temporários (/tmp)

➡ Banco implica custo de persistência e manutenção.

6 quando o custo operacional não se justifica

Cenário

- Projeto pequeno
- Ambiente acadêmico
- Aplicação local simples

Exemplo

- Trabalho de faculdade
- Protótipo rápido
- Ferramenta interna simples

Melhor alternativa

- Arquivos

➡ Banco traz:

- Instalação
 - Configuração
 - Backup
 - Manutenção
-

7 quando você precisa de controle total do formato

Cenário

- Layout de dados muito específico
- Integração com hardware
- Compatibilidade binária

Exemplo

- Emuladores
- Drivers

- Sistemas legados

Melhor alternativa

- Arquivo binário estruturado
-

Regra prática de decisão sobre armazenamento de dados em arquivos

Faça estas perguntas:

- Tenho muitos usuários ao mesmo tempo?
- Preciso de consultas complexas?
- Preciso de integridade transacional?
- Preciso escalar?

Se a maioria for **não**, **não use banco de dados**.

Resumindo, não use banco de dados quando os dados são simples, locais, sem concorrência, de baixo volume ou quando o custo e a complexidade superam os benefícios.

BUFFER

Buffer pode ser usado tanto em arquivos sequenciais quanto aleatórios.

Um **buffer** é uma **área temporária de memória usada para armazenar dados enquanto eles estão sendo transferidos entre dois lugares**.

A ideia principal é **evitar que um dispositivo ou programa mais rápido tenha que esperar por outro mais lento**.

Exemplo:

Pense no buffer como uma **bandeja** em um restaurante:

- A cozinha prepara vários pratos e coloca na bandeja.
- O garçom leva a bandeja até a mesa.
- Assim, a cozinha não precisa esperar o garçom a cada prato.

A **bandeja é o buffer**.

Exemplo em computação

Quando um programa grava dados em um arquivo:

1. Os dados são colocados primeiro no **buffer (memória)**.
2. Depois são enviados **de uma vez para o disco**.

Isso é mais eficiente porque o **disco é mais lento que a memória**.

Exemplo em Java

```
BufferedWriter bw = new BufferedWriter(new FileWriter("dados.txt"));
```

Nesse caso:

- **FileWriter** → envia dados para o arquivo.
- **BufferedWriter** → usa um **buffer na memória** para juntar vários dados antes de gravar.

Resultado: **menos acessos ao disco e maior desempenho**.

Resumindo

Um **buffer**:

- é um **armazenamento temporário**
- fica geralmente na **memória RAM**
- **acumula dados antes de enviar ou receber**
- melhora o **desempenho de leitura e escrita**

Um exemplo bem comum

- **Streaming de vídeo** (YouTube, Netflix) usa buffer para carregar alguns segundos do vídeo antes de reproduzir 

Assim o vídeo **não trava enquanto os dados chegam da internet**.

Arquivo aleatorio

Exemplo prático em Java

Usamos a classe `RandomAccessFile`.

Exemplo 1 – Ler um byte específico

```
RandomAccessFile raf = new RandomAccessFile("dados.dat", "r");
```

```
// vai para o byte 5
raf.seek(5);
int valor = raf.read();
System.out.println(valor);
raf.close();
```

-  O programa lê diretamente o byte 5.

Exemplo 2 – Acesso a registro fixo

Suponha:

- Cada registro tem 20 bytes
- Você quer o registro 3

```
int tamanhoRegistro = 20;
```

```
int numeroRegistro = 3;
```

```
RandomAccessFile raf = new RandomAccessFile("dados.dat", "r");
```

```
// posiciona no início do registro 3
```

```
raf.seek(numeroRegistro * tamanhoRegistro);
```

```
// agora lê o registro
```

```
byte[] buffer = new byte[tamanhoRegistro];
```

```
raf.read(buffer);
```

```
raf.close();
```

O que acontece internamente?

seek(posicao):

1. Move o ponteiro para posicao bytes a partir do início
2. A próxima operação começa dali

Importante:

seek(0) → início do arquivo

seek(10) → décimo byte

Observação

seek() trabalha com:

offset em bytes, não em registros, linhas ou campos.

Por isso fazemos:

posição = númeroRegistro × tamanhoRegistro

Diferença entre sequencial e aleatório

Não utiliza seek(), pois o acesso é sequencial:

leitura → leitura → leitura → leitura

Utiliza seek() para acessar o registro desejado.

salta direto para posição desejada

Resumo

seek() reposiciona o ponteiro do arquivo para um byte específico, permitindo acesso direto a qualquer parte do arquivo.